



Clean Code

Filosofía del Clean Code (Código Limpio)

El código limpio no es solo un conjunto de reglas estéticas, sino una filosofía de desarrollo orientada a la sostenibilidad del proyecto. Como afirma Robert C. Martin, "el código limpio siempre parece que fue escrito por alguien a quien le importa".

- **Nombres Claros:**

La claridad en el nombrado es el primer paso hacia la autodocumentación del código.

Evitar la Desinformación: No usar nombres que sugieran algo distinto al contenido real (ej. evitar `lista_array` si el tipo de datos cambia en el futuro).

Nombres del Dominio: Se deben preferir términos que pertenezcan al negocio o al problema que se está resolviendo, en lugar de términos puramente técnicos.

Ejemplo en Java:

- **Baja calidad:** `int d; // días`
- **Alta calidad:** `int diasTranscurridos;`

- **Funciones pequeñas:**

Para que una función sea mantenible, debe ser pequeña y realizar **una sola cosa** (un solo nivel de abstracción).

Manejo de Errores: Las funciones deben preferir lanzar excepciones en lugar de retornar códigos de error como `-1`, `null` o `false`, ya que estos últimos obligan al llamador a gestionar la lógica de error inmediatamente, ensuciando el flujo principal.

Silencio de Errores: Es una práctica prohibida ocultar errores con bloques `catch` vacíos, ya que invisibilizan fallos que podrían ser críticos en producción.

- **Manejo de Errores:** Preferir lanzar excepciones que retornar códigos de error (`-1`, `null` o `false`). No ocultar errores con `catch` vacíos.



- **Cohesión y Acoplamiento:**

El diseño robusto depende de cómo se relacionan los componentes del sistema.

- **El Equilibrio de Dependencias**

- **Alta Cohesión:** Los elementos dentro de un mismo módulo o clase deben estar estrechamente relacionados entre sí en cuanto a su funcionalidad.

- **Bajo Acoplamiento:** Los módulos deben depender lo mínimo posible entre sí. Esto permite que un cambio en una clase no genere un "efecto dominó" que rompa otras partes del sistema.

- **Gestión de Límites (Código de Terceros)**

Integrar librerías externas es un punto crítico de fragilidad. Se recomienda el uso de **"Wrappers"** o capas de abstracción.

- **Propósito:** Si la librería externa cambia su API o es reemplazada, solo debemos modificar nuestro "Wrapper", protegiendo al resto del sistema de cambios disruptivos.
- **Límites:** Cómo integrar código de terceros (librerías). Se recomienda crear "Wrappers" o capas de abstracción para que un cambio en la librería externa no rompa todo nuestro sistema.



Refactoring Seguro: Técnicas frecuentes

Definido por **Martin Fowler**.

Cambiar la estructura interna del código **sin cambiar su comportamiento**.

Regla fundamental: **refactorizar con tests**.

Técnicas importantes:

- **Extract Method:** Sacar un fragmento de código de un método largo y convertirlo en un método nuevo con un nombre descriptivo. Mejora la legibilidad instantáneamente.
- **Move Method:** Si un método usa más características de otra clase que de la propia donde está definido, debe moverse a esa otra clase.
- **Introduce Parameter Object:** Si un grupo de parámetros siempre viaja junto (ej: `fechaInicio`, `fechaFin`), se reemplazan por un solo objeto (ej: `RangoFechas`). Reduce la complejidad de las firmas de los métodos.

Code review: checklist, criterios de aceptación técnica, feedback profesional.

El code review no es una auditoría ni una crítica personal; es una práctica colaborativa para mejorar la calidad del software.

- Checklist de Revisión:
No intentes revisarlo todo a la vez. Divide tu atención en estas categorías:
 - **Lógica y Funcionalidad:** ¿El código hace lo que se supone que debe hacer? ¿Cubre casos de borde (edge cases)?
 - **Legibilidad:** ¿Los nombres de variables y funciones son descriptivos? ¿Podría un desarrollador nuevo entender esto sin preguntar?
 - **Diseño y Arquitectura:** ¿Sigue los principios **SOLID** o **DRY**? ¿Está en la capa correcta del proyecto?
 - **Seguridad:** ¿Hay datos sensibles expuestos? ¿Es vulnerable a inyecciones o ataques comunes?
 - **Rendimiento:** ¿Hay bucles innecesarios o consultas a base de datos ineficientes?



- Criterios de Aceptación Técnica (TAC)

Son **condiciones técnicas** que el código debe cumplir para ser aceptado e integrado al proyecto.

Conjunto de **requisitos técnicos** que el código debe cumplir antes de ser **aprobado**, relacionados con calidad, arquitectura, seguridad, rendimiento y mantenibilidad.

Qué suelen incluir

Ejemplos típicos de TAC en un proyecto:

- ☒ El código **compila y pasa todos los tests**.
- ☒ Se respetan **principios de diseño** (por ejemplo SOLID).
- ☒ Cumple con los **estándares de código del proyecto** (naming, estilo, etc.).
- ☒ Incluye **tests unitarios o de integración** si corresponde.
- ☒ No introduce **bugs ni rompe funcionalidades existentes**.
- ☒ Manejo correcto de **errores y excepciones**.
- ☒ No genera **deuda técnica innecesaria**.
- ☒ Cumple con **seguridad y buenas prácticas**.

Antes de dar el "Approve", el código debería cumplir con ciertos mínimos innegociables:

Criterio	Descripción
Pruebas (Tests)	Debe incluir tests unitarios o de integración que pasen correctamente.
Estilo (Linting)	Debe cumplir con la guía de estilo del equipo (formateo, indentación).
Documentación	Si hay lógica compleja o nuevos endpoints, deben estar documentados (JSDoc, Swagger, README).
Atomicidad	El PR (Pull Request) debe resolver una sola cosa. Si es demasiado grande, es difícil de revisar.

- **Feedback Profesional:** El feedback debe ser **constructivo y respetuoso**, enfocado en mejorar el código, no en criticar a la persona.

Buenas practicas:

- Criticar el código, no al desarrollador
- Explicar el motivo
- Proponer alternativas
- Reconocer lo positivo